

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	__DR Sukanya ledalla__
Student Name	__A M P S SUBRAHMANYA SREEPAD__
Roll Number	__2520030152__
Branch	CSE
Course Outcome	5

Work Done Summary

In this case study, we analyzed Reliance Jio call-detail-record (CDR) duration sorting using Bucket Sort for skewed distributions.

The work focused on:

1. Sorting large-scale call durations efficiently.
2. Dividing duration values into fixed-size buckets.
3. Distributing elements into appropriate buckets.
4. Sorting each bucket independently.
5. Concatenating buckets to obtain final sorted order.
6. Handling highly skewed telecom data distributions.
7. Comparing bucket sort with counting sort and radix sort.
8. Studying adaptive bucket sizing techniques for performance improvement.

Implementation Details

1. Considered the input array:

[37, 12, 78, 25, 91, 4, 56, 8, 33, 67, 19, 45]

2. Created 10 buckets representing ranges:

- 0–9
- 10–19
- 20–29
- ...
- 90–99

3. Calculated bucket index using:

```
bucketIndex = value / 10
```

4. Distributed elements into buckets:

- Bucket 0 → [4, 8]
- Bucket 1 → [12, 19]
- Bucket 2 → [25]
- Bucket 3 → [37, 33]
- Bucket 4 → [45]
- Bucket 5 → [56]
- Bucket 6 → [67]
- Bucket 7 → [78]
- Bucket 9 → [91]

5. Sorted each bucket individually using insertion sort.

6. Concatenated buckets sequentially.

7. Generated the final sorted output.

8. Explained expected $O(n)$ performance for uniform distributions.

9. Discussed worst-case $O(n^2)$ behavior when all elements fall into one bucket.

10. Analyzed adaptive bucket sizing for skewed telecom workloads.

RESULTS:SCREENSHOTS/Output:

Input Array:

[37, 12, 78, 25, 91, 4, 56, 8, 33, 67, 19, 45]

Bucket Distribution:

Bucket 0 (0-9):

[4, 8]

Bucket 1 (10-19):

[12, 19]

Bucket 2 (20-29):

[25]

Bucket 3 (30-39):

[33, 37]

Bucket 4 (40-49):

[45]

Bucket 5 (50-59):

[56]

Bucket 6 (60-69):

[67]

Bucket 7 (70-79):

[78]

Bucket 8 (80-89):

[]

Bucket 9 (90-99):

[91]

Final Sorted Array:

[4, 8, 12, 19, 25, 33, 37, 45, 56, 67, 78, 91]

Bucket Sort Completed Successfully.

Conclusion

In this case study, Bucket Sort was used because it efficiently handles large datasets with approximately uniform numeric distributions such as call durations.

The algorithm divided values into multiple buckets, sorted each bucket independently, and concatenated the results to obtain the final sorted order. This reduced sorting overhead and improved scalability for telecom billing workloads.

For highly skewed real-world Jio CDR distributions, adaptive bucket sizing and logarithmic bucket spacing further improve performance by preventing excessive bucket imbalance.

This approach is highly suitable for large-scale analytics systems, telecom billing platforms, streaming logs, and distributed data-processing environments where fast numeric sorting is required.

Github Link: <https://github.com/ampssubrahmanyasreepad-svg/DSA-CaseStudies-correct>

Student Signature	Faculty Signature